

Automatic Detection and Compilation of Structured Tensors

by

Selena C. Qiao

B.S. in Computer Science and Engineering, Massachusetts Institute of Technology, 2026.

Submitted to the Department of Electrical Engineering and Computer Science
in partial fulfillment of the requirements for the degree of

MASTER OF ENGINEERING IN ELECTRICAL ENGINEERING AND
COMPUTER SCIENCE

at the

MASSACHUSETTS INSTITUTE OF TECHNOLOGY

May 2026

© 2026 Selena C. Qiao. All rights reserved.

The author hereby grants to MIT a nonexclusive, worldwide, irrevocable, royalty-free license to exercise any and all rights under copyright, including to reproduce, preserve, distribute and publicly display copies of the thesis, or release the thesis under an open-access license.

Authored by: Selena C. Qiao
Department of Electrical Engineering and Computer Science
May 8th, 2026

Certified by: Alan Edelman
Professor of Applied Mathematics, Thesis Supervisor

Accepted by: Katrina LaCurts
Chair
Master of Engineering Thesis Committee

Automatic Detection and Compilation of Structured Tensors

by

Selena C. Qiao

Submitted to the Department of Electrical Engineering and Computer Science
on May 8th, 2026 in partial fulfillment of the requirements for the degree of

MASTER OF ENGINEERING IN ELECTRICAL ENGINEERING AND
COMPUTER SCIENCE

ABSTRACT

Tensors are a central abstraction in modern scientific computing, spanning high-performance computing, PDE solvers, and machine learning. In practice, many tensors exhibit mathematical structure – such as symmetry, triangularity, or sparsity – that can be exploited to reduce computation and memory cost. However, existing approaches typically require explicit programmer annotations or rely on domain-specific languages, limiting usability and reliability.

We present a compiler framework for the automatic detection and optimization of structured tensors. Built on MLIR, our approach leverages a semantically rich intermediate representation to infer structural properties directly from high-level tensor operations. We define detection and propagation rules to track structure throughout a program and use this information to select structure-aware implementations during code generation.

Our system eliminates the need for manual intervention while enabling consistent performance improvements, bridging the gap between high-level tensor abstractions and optimized low-level execution.

Thesis supervisor: Alan Edelman

Title: Professor of Applied Mathematics

Contents

<i>List of Figures</i>	7
<i>List of Tables</i>	9
1 Introduction	11
1.1 MLIR	11
1.2 StableHLO	12
1.3 LAPACK and BLAS	12
1.4 Enzyme and Reactant	13
1.5 Related Work	13
1.5.1 TACO	13
1.5.2 Finch	13
1.5.3 Limitations	14
1.6 Overview of the Structured Tensors Compiler	14
2 Detection	15
2.1 Detection Algorithm	16
2.1.1 Supported Structured Tensors	16
2.2 Detection Rules by Structure	17
2.2.1 Symmetric	17
2.2.2 Triangular	18
2.2.3 Tridiagonal	19
3 Optimization	21
3.1 Supported Optimizations	21
3.2 Specialized Kernels	22
3.3 Mathematical Simplifications	22
3.4 Lowering of Custom Calls	23
3.5 Layout Optimizations	23
3.5.1 <code>symm</code> and <code>trmm</code>	23
3.5.2 <code>syrk</code>	24
3.5.3 <code>gtsv</code>	24
3.6 Fusion of Addition and Multiplication	24

4	Evaluation	27
4.1	Benchmarks	27
4.1.1	Basic Benchmarks	27
4.1.2	Optimizing the Newton-Schulz Iteration	28
5	Conclusion	35
A	Full MLIR program for Newton-Schulz	37
B	symm Lowering	39
	<i>References</i>	43

List of Figures

1.1	Example of StableHLO program	12
2.1	MLIR construction of triangular matrix (data types omitted)	18
2.2	MLIR construction of tridiagonal matrix (data types omitted)	19
2.3	An example of an MLIR program after a detection pass. Since <code>dot_general</code> can be optimized for symmetric matrices, the compiler tries to infer whether its arguments are symmetric and annotates them accordingly. Because <code>arg1</code> is an input to the function, its structural properties are unknown.	19
3.1	MLIR program using <code>symm</code> before fusion pass	25
3.2	MLIR program using <code>symm</code> after fusion pass	25
4.1	Benchmark for triangular matrices	28
4.2	MLIR for triangular tensor benchmark after compiler passes.	28
4.3	Benchmark for symmetric matrices	29
4.4	Implementation of the Newton-Schulz Iteration in Julia	29
4.5	Newton-Schulz MLIR after detection and optimization passes, but before custom call lowering. Variables and constants have been renamed to better match what they represent. The compiler was able to successfully annotate symmetric matrices with <code>enzymexla.symmetric_matrix = [#enzymexla<guaranteed GUARANTEED>]</code> , which then allows the compiler to make the necessary rewrites into <code>symm</code> and <code>syrk</code>	31
4.6	Newton-Schulz kernel statistics without structured tensors detection. All matrix multiplications use the <code>gemm</code> (general matrix multiplication) kernel.	31
4.7	Newton-Schulz kernel statistics with structured tensors detection and optimization enabled. This run includes the call to <code>symm</code> ; however since on this architecture <code>symm</code> is more expensive than <code>gemm</code> it would not replace <code>gemm</code> in practice.	32
4.8	speed-up of structured tensor benchmarks over the default optimization options across different sizes of input matrices.	32
4.9	Relative speed-up of each benchmark relative to the default optimization options, with normalized default runtime.	33
B.1	<code>symm</code> call to be lowered	40
B.2	<code>symm</code> call lowered for CPU	41
B.3	<code>symm</code> call lowered for CUDA	42

List of Tables

3.1 Supported optimizations by tensor structure	21
---	----

Chapter 1

Introduction

Tensors are frequently used in modern scientific computing, from high performance computing to PDE solvers and machine learning. Oftentimes they are used to store large amounts of data. In finite element analysis, they’re used to represent stiffness or mass matrices; in quantum physics, density matrices; in climate modeling, pressure and velocity fields, etc. Tensors are also often used to represent operators, such as block-structured operators, used in physical systems modeling to couple multiple physical phenomena.

In practice, a lot of tensors that we encounter are structured. The authors of Finch, a related framework for computing with sparse tensors, describe structured tensors as tensors which contain patterns that allows us to optimize storage or computation [1]. For instance, a structured tensor can be symmetric, banded, sparse, triangular, etc.

Many tensor computations exhibit latent mathematical structure—such as symmetry or triangularity—that can be leveraged to reduce computational cost. However, existing systems typically require explicit user annotations and domain-specific languages (DSLs), or rely on low-level analyses that fail to recover such structure reliably.

Our approach instead integrates structure-aware reasoning directly into the compilation pipeline. We aim to enable automatic detection and optimization of structured tensors without any additional effort from the user.

1.1 MLIR

To be able to run the compiler on existing programs, the compiler needs to be language-independent. We achieve this by operating on top of Multi-Level Intermediate Representation, or MLIR.

MLIR is a compiler infrastructure from the LLVM project that lets users define different dialects of IR, which are collections of related types and operations that represent a specific abstraction level [2]. These dialects can then be lowered to another dialect; in other words, it is transformed to a lower level of abstraction. This allows preservation of semantics at many different levels.

There are two big benefits of operating on top of MLIR. The first is that this compiler can integrate with any language or framework that can be lowered to MLIR, such as Python (including PyTorch), Swift, Julia, etc [3–5]. This means that this compiler is a general

```

func.func @main(%arg0: tensor<2x2xf32>) -> tensor<2x2xf32> {
  %0 = stablehlo.transpose %arg0, dims = [1, 0] : (tensor<2x2xf32>) -> tensor<2x2xf32>
  %1 = stablehlo.dot_general %0, %arg0, contracting_dims = [1] x [0],
    precision = [DEFAULT, DEFAULT] : (tensor<2x2xf32>, tensor<2x2xf32>)
    -> tensor<2x2xf32>
  %2 = stablehlo.transpose %1, dims = [1, 0] : (tensor<2x2xf32>) -> tensor<2x2xf32>
  %3 = stablehlo.add %1, %2 : tensor<2x2xf32>
  return %2 : tensor<2x2xf32>
}

```

Figure 1.1: Example of StableHLO program

compiler – it is not bound to any specific language. This provides a huge benefit to the user by removing the need to port code from one language to another, or learning a new domain-specific language.

While the compiler itself is not language-specific, for it to work end-to-end for a given language, a language-specific front-end needs to be written. In this project, this compiler was integrated with the Julia front-end for Enzyme. In the future it can also integrate with other languages that compile to MLIR.

The second benefit of MLIR is that it enables the compiler to preserve tensor algebra semantics. MLIR dialects can capture high level operations, like matrix multiplication or LU factorization, directly in the IR of a program. This allows for more comprehensive automatic detection of structures without having to do more complex analyses that require tracing through the data and control flow of a program in order to reconstruct these higher level semantics.

1.2 StableHLO

The primary dialect that this compiler operates on is the StableHLO dialect. StableHLO is an operation set for high-level operations (HLO), primarily designed for machine learning frameworks [6]. It provides a portable, stable interface between frameworks like JAX, PyTorch, and TensorFlow and compilers like this one.

Figure 1.1 shows an example of a simple StableHLO program that performs matrix multiplication of an input and its transpose, then adds the result to the transpose. These are generally the types of input programs that the compiler will operate on.

1.3 LAPACK and BLAS

Similar to previous structure-aware compilers, our optimizations take advantage of standardized high-performance numerical linear algebra libraries such as BLAS and LAPACK [7]. BLAS provides a collection of low-level, highly optimized kernels for basic vector and matrix operations (e.g., matrix–vector and matrix–matrix multiplication), while LAPACK builds on these primitives to implement higher-level algorithms for solving linear systems, eigenvalue problems, and matrix factorizations. For CUDA architectures, our compiler uses cuBLAS,

cuSPARSE, and cuSOLVER, the NVIDIA CUDA implementations for BLAS and LAPACK.

These libraries are engineered to exploit hardware features such as cache hierarchies, vectorization, and parallelism, and feature specialized structure-specific kernels. For example, operations involving symmetric or triangular matrices can avoid redundant computation and reduce memory traffic compared to their dense counterparts. Our compiler attempts to automatically map these linear algebra operations to the appropriate specialized implementations.

1.4 Enzyme and Reactant

Enzyme is an automatic differentiation compiler that operates on top of MLIR and LLVM [8]. Its primary purpose is to allow programmers to take derivatives of their code without additional efforts.

While this project does not rely on or provide any automatic differentiation capabilities, it integrates into the Enzyme compiler in order to utilize the existing framework for working with MLIR and Julia.

Reactant is a tracing framework for Enzyme, which takes Julia functions and lowers them into MLIR, making the program available for MLIR-based compiler passes and optimizations [9, Section 2.2]. Together, these two repositories create a solid foundation upon which this compiler functions.

1.5 Related Work

1.5.1 TACO

TACO is one example of a compiler developed to address the issue of automatically generating optimized code for sparse tensors [10]. It allows users to explicitly specify the structure of a sparse tensor in their code in a domain-specific language, then uses this sparsity information to compile their code into optimized kernels.

TACO aims to solve the problem of handwritten kernels, which can become increasingly complicated when more than one tensor is sparse, and their sparsity patterns compose. Programmers can use TACO to generate code by invoking a web or command line tool or using an interactive command line workflow; in other words, TACO is an external tool, rather than built into the compiler itself.

1.5.2 Finch

Finch is another DSL that aimed to improve upon TACO, and uses control and data flow analysis to detect tensor structure [1]. It can be used more seamlessly to express entire algorithms.

Finch represents control flow patterns using looplets, which represent the dependencies for any given data structure; it then uses these looplets to analyze how tensors are populated in the program. It can detect sparsity patterns, blocked data (where non-zero elements appear in blocks), and runs of repeated values, but cannot as easily make inferences from high level operations and cannot detect value properties.

1.5.3 Limitations

These previous compilers are limited in the amount of automatic detection possible, and require domain-specific languages to function. TACO requires programmers to manually declare structural properties in order to perform any optimizations. This can be a tedious, time-consuming, and error prone process, which then may need to be repeated in some capacity if the program changes. Finch has some automatic detection, but can only detect structure from strict control flow analysis, which means it cannot detect value properties such as symmetry, either through construction or concrete values. In order for such properties to be specified in Finch, the programmer must manually declare it. Another large limitation is that these compilers all use DSLs, which means in order to use them, a programmer needs to port all their code into the new language, which can be very cumbersome.

1.6 Overview of the Structured Tensors Compiler

This work presents a compiler framework for the automatic detection and optimization of structured tensors in high-level programs. The compiler performs two core tasks: (1) structure detection and propagation, and (2) optimization.

Detection is carried out over MLIR programs, which preserves high-level tensor operations and enables direct reasoning about their mathematical properties. In this phase, the compiler annotates any detected structural properties and propagates them through the rest of the program. Once identified, structural information is used to guide optimization and code generation. Similar to previous compilers, for each supported property, the compiler defines a set of operations that can be lowered to specialized kernels. This design enables modular extension to additional structures and optimizations.

By combining high-level semantic analysis with targeted lowering, our compiler automatically discovers and exploits structure in tensor programs, achieving performance improvements without requiring additional efforts from the programmer. The following sections will provide an overview of the detection and optimization phases respectively, as well as an evaluation of performance improvements.

Chapter 2

Detection

The first goal of the compiler is to detect when a tensor is structured. Our approach is to detect these structural properties as they are mathematically constructed and propagated through the program. This is made much easier by the fact that the compiler is built on top of MLIR, which allows us to capture a lot of the higher-level mathematical semantics of the program (rather than having to reconstruct these operations through costly data flow analyses, for example).

In general, these structures can arise in three different ways:

1. **Concrete constant values:** When a constant tensor is constructed in the program, the numerical entries are available at compile-time. The compiler can therefore simply examine the concrete values of the tensor for any structural patterns.
2. **Mathematical construction:** Certain combinations of tensor operations can guarantee structural properties in the resulting tensor. For instance, the result of $A + A^T$ is always guaranteed to be symmetric.
3. **Propagation of existing structure:** Once a tensor is determined to have a certain structure, it may propagate this structure throughout the rest of the program. For instance, the sum of two symmetric tensors is guaranteed to also be symmetric.

We define more specific rules within these three broad categories for each type of structure that we wish to detect within the compiler. The specific structural properties that we wish to detect and the corresponding detection rules will be described more thoroughly later in this section.

For a given structural property, we also define a set of operations that can be optimized if we know that its operands have the property. For instance, say we have a specialized kernel for matrix multiplication (`DotGeneral`) given that one matrix is symmetric; then, the `DotGeneral` operation should be added to the set of optimizable operations for symmetric tensors.

Once we have defined these sets of rules and operations, the compiler will perform partial scans of the MLIR of a given program for structural properties that could result in optimizations and annotate the IR as it traverses it. These scans are triggered from the compiler’s optimization passes; they only occur when the compiler encounters an operation

in a structure’s optimizable set. The following section will describe the detection algorithm in greater detail.

2.1 Detection Algorithm

To detect structure properties, the compiler traverses the IR in a backwards manner. Each structured tensor has a set of operations that can potentially be optimized. For each such operation, we define a rewrite pattern that is triggered when the compiler encounters the operation; the rewrite pattern will then call the detection code to determine whether or not the operands contain the necessary structure properties for the optimization to occur.

The detection algorithm will first check its value cache and examine the IR to determine if the queried structural property of the given matrix is already known. If so, it will simply return the result from the cache.

If the structure of the given tensor is dependent on the structure of the operands, the compiler will add the operands to the queue. This allows the compiler to trace upwards through the parent operations of the operands in order to determine whether the property holds.

The detection is performed in this manner instead of a full forward detection pass because a forward pass is more expensive, as we would need to iterate through the entire IR. Furthermore, as different optimization passes continuously make changes to the IR, a full forward detection pass may quickly become outdated. Detecting structure through a backward pass is cheaper, and as the IR is updated by other optimizations, a partial forward pass can again be initiated from the point of mutation. This allows for more flexibility and is more efficient than repeatedly running full forward passes.

2.1.1 Supported Structured Tensors

We make a distinction between sparsity patterns and value properties. A tensor has a sparsity pattern if there is a pattern of indices where the entries at those indices are zero. The sparsity patterns that may be detected for a $n \times n$ matrix include:

$$\begin{aligned}
\text{Dense} &= \{(i, j) : 1 \leq i, j \leq n\} \\
\text{Band}(k_l, k_u) &= \{(i, j) : -k_l \leq j - i \leq k_u\} \\
\text{UpperTri} &= \{(i, j) : i \leq j\} = \text{Band}(0, n - 1) \\
\text{LowerTri} &= \{(i, j) : i \geq j\} = \text{Band}(n - 1, 0) \\
\text{Tridiag} &= \{(i, j) : |i - j| \leq 1\} = \text{Band}(1, 1) \\
\text{Bidiag} &= \{(i, j) : 0 \leq j - i \leq 1\} = \text{Band}(0, 1) \\
\text{Diag} &= \{(i, j) : i = j\} = \text{Band}(0, 0)
\end{aligned}$$

A tensor has a value property if there exists a relationship between the matrix entries.

The value properties that may be detected for an $n \times n$ matrix are

$$\begin{aligned}\text{UnitDiag}(A) &\Leftrightarrow A_{ii} = 1 \text{ for all } i \\ \text{Symmetric}(A) &\Leftrightarrow A_{ij} = A_{ji} \text{ for all } i, j \\ \text{Hermitian}(A) &\Leftrightarrow A_{ij} = \overline{A_{ji}} \text{ for all } i, j\end{aligned}$$

The full set of structured matrix types are elements of the product lattice:

$$\mathcal{M} = \mathcal{S} \times \mathcal{P}(\{\text{UnitDiag}, \text{Symmetric}, \text{Hermitian}\})$$

where $\mathcal{P}(\cdot)$ denotes the power set. In other words, a structured tensor may have a sparsity pattern and any number of value properties.

Ideally, this compiler would be able to detect all structured matrix types in this set. This project leaves out generic banded matrices, bidiagonal matrices, and Hermitian matrices, simply because the other structures were projected to result in stronger performance improvements and appear more frequently in programs. In the future, these excluded properties can easily be implemented using the same framework.

2.2 Detection Rules by Structure

2.2.1 Symmetric

A constant matrix A is symmetric if A is square and $A_{ij} = A_{ji}$ for all i, j . A constant matrix is also symmetric if it is the result of broadcasting a single constant value (i.e. if all of the elements are the same). We distinguish this from the first because it appears differently in the IR of the program.

If some operation op is commutative and elementwise, then $\text{op}(A, A^T)$ is guaranteed to be symmetric. For example, $A + A^T$ is symmetric, $A \oplus A^T$ is symmetric, etc. Similarly, AA^T is guaranteed to be symmetric.

Diagonal matrices are also guaranteed to be symmetric. We can generally detect the construction of diagonal matrices through StableHLO's `ScatterOp`, which can distribute a vector of values along the diagonal of a matrix. If that matrix is originally all zeroes, then the result of this operation is a diagonal matrix and therefore also symmetric.

A matrix is also symmetric if it is the result of an `syk` operation (see the optimization section for a more detailed description on `syk`).

If an operation is elementwise and both of its operands are symmetric, the result of the operation is also guaranteed to be symmetric.

Factoring of Multiplication

To further facilitate the detection of symmetric matrices generated from matrix multiplication, we also rewrite operations in the form $(a * \text{op}(Z)) \times (b * \text{op}(Z))$ to $(a * b) * (\text{op}(Z) \times \text{op}(Z))$. If either operand is not the result of a `MultiplyOp` we simply set the corresponding coefficient to 1. This simplifies the detection step and reduces the number of elementwise multiplications.

2.2.2 Triangular

The detection for triangular matrices is a little more complex because we are interested in both whether it is upper or lower triangular, and additionally whether the diagonal is a unit diagonal or not. Therefore, the detection pass for triangular matrices involves up to four separate detection passes that are dependent on each other, depending on the operation being optimized:

- **Upper triangular:** dependent on lower triangular and upper unit triangular
- **Lower triangular:** dependent on upper triangular and lower unit triangular
- **Upper unit triangular:** dependent on upper triangular and lower unit triangular
- **Lower unit triangular:** dependent on lower triangular and upper unit triangular

The reason these separate passes depend on each other is because an upper triangular matrix can easily be converted to a lower triangular matrix through a transpose operation and vice versa; similarly, an upper unit triangular matrix can be constructed through a `ScatterOp` to the diagonal of an upper triangular matrix, etc. The missing dependencies between properties is because a matrix cannot transform into that structure without passing through another (i.e. an upper triangular matrix cannot become a lower unit triangular matrix without first becoming a lower triangular matrix or an upper unit triangular matrix).

The primary construction of triangular matrices that the compiler detects is the following:

```
1 %cst_0 = stablehlo.constant dense<0> : tensor<2x2>
2 %0 = stablehlo.iota dim = 0 : tensor<2x2>
3 %1 = stablehlo.iota dim = 1 : tensor<2x2>
4 %2 = stablehlo.compare LE, %0, %1 : (tensor<2x2>, tensor<2x2>) -> tensor<2x2>
5 %3 = stablehlo.select %2, %arg0, %cst_0 : tensor<2x2>, tensor<2x2>
```

Figure 2.1: MLIR construction of triangular matrix (data types omitted)

In this MLIR snippet, two `Iota` operations construct square matrices that increase along their row and column dimensions, respectively. Then, the `Compare` operation constructs a mask that selects all elements where the row-iota tensor is less than or equal to the column-iota tensor, which results in a mask that selects the upper triangular of the matrix, including the diagonal. Then, the `Select` operation uses the upper triangular mask to select that portion of the matrix. The detection for triangular matrices looks for variations of this pattern – i.e. it is possible for different comparison options to be used, or `iota` dimensions to be swapped, and result in a different triangular matrix, etc.

Constant matrices are upper triangular if $A(i, j) = 0$ for $i > j$, etc., and unit triangular if $A(i, i) = 1$ for all $0 \leq i < n$.

Like symmetric matrices, triangular matrices can also arise as the result of `Scatter` operations.

Triangular matrices can propagate through unary operations `op` where `op(0)=0`, and additionally `op(1) = 1` for unit triangular matrices (for instance, `Sqrt`). Similarly, they

```

1 %c = stablehlo.constant dense<[[0, 0], [1, 1], [2, 2], [1, 0], [2, 1], [0, 1], [1,
  2],]> : tensor<28x2>
2 %cst = stablehlo.constant dense<0> : tensor<10x10>
3 %0 = stablehlo.concatenate %arg1, %arg0, %arg2, dim = 0: (tensor<10>, tensor<9>,
  tensor<9>) -> tensor<28>
4 %1 = stablehlo.scatter(%cst, %c, %0) <...> ({(%arg4, %arg5):stablehlo.return %arg5
  }) : (tensor<10x10>, tensor<28x2>, tensor<28>) -> tensor<10x10>

```

Figure 2.2: MLIR construction of tridiagonal matrix (data types omitted)

```

1 func.func @main(%arg0: tensor<256x256xf64> {enzymexla.memory_effects = []}, %arg1:
  tensor<256x256xf64> {enzymexla.memory_effects = []}) -> tensor<256x256xf64>
  attributes {enzymexla.memory_effects = []} {
2   %0 = stablehlo.transpose %arg0, dims = [1, 0] : (tensor<256x256xf64>) -> tensor
    <256x256xf64>
3   %1 = stablehlo.add %0, %arg0 {enzymexla.symmetric_matrix = [#enzymexla<
    guaranteed GUARANTEED>]} : tensor<256x256xf64>
4   %2 = stablehlo.dot_general %1, %arg1, contracting_dims = [0] x [0], precision =
    [DEFAULT, DEFAULT] : (tensor<256x256xf64>, tensor<256x256xf64>) -> tensor
    <256x256xf64>
5   return %2 : tensor<256x256xf64>
6 }

```

Figure 2.3: An example of an MLIR program after a detection pass. Since `dot_general` can be optimized for symmetric matrices, the compiler tries to infer whether its arguments are symmetric and annotates them accordingly. Because `arg1` is an input to the function, its structural properties are unknown.

can propagate through binary elementwise operations where $\text{op}(0, 0) = 0$, and additionally $\text{op}(1, 1) = 1$ for unit triangular matrices, for instance elementwise multiplication. Additionally, the product of two triangular matrices is also triangular.

2.2.3 Tridiagonal

A constant matrix is tridiagonal if $\{A(i, j) : |i - j| > 1\} = 0$.

We primarily detect the following construction pattern for tridiagonal matrices:

The `Scatter` operation distributes the input vectors `arg1`, `arg2`, and `arg3` along the three diagonals of the zero tensor.

Tridiagonal matrices, like triangular matrices, can be propagated through unary operations `op` where $\text{op}(0)=0$ and binary elementwise operations where $\text{op}(0, 0) = 0$.

Chapter 3

Optimization

Once a tensor is determined to have a certain structure, there are a number of optimizations that can become possible, based on both the detected structure and the operations in which the tensor is involved in. These optimizations can roughly be split into two types: the first is to forward certain operations to specialized kernels optimized for the detected structure (i.e. in the LAPACK library). For instance, say a program performs general matrix multiplication between two matrices A and B . If the compiler determines that A is a symmetric matrix, it can convert the `DotGeneral` call into a `Symm` call, which is a specialized matrix multiplication kernel for symmetric matrices. This approach is also used in previous work, such as TACO, and simply requires integrating these custom calls into the compiler.

The second type of optimization is a mathematical simplification of high level operations. Given knowledge of a tensor’s structure, some operations can be simplified or completely removed from the IR; for instance, if a program takes the transpose of a tensor we know is symmetric, that transpose operation is a no-op and can be removed from the IR.

3.1 Supported Optimizations

This table provides an overview of the supported optimizations for each type of structure, where the first row contains specialized kernels, and the second row contains mathematical simplifications.

	Symmetric	Triangular	Tridiagonal
Opcalls	<code>symm</code> , <code>syrk</code>	<code>trmm</code>	<code>gtsv</code>
Simplifications	<code>TransposeOp</code>	<code>LUFactorizationOp</code> (upper only)	

Table 3.1: Supported optimizations by tensor structure

3.2 Specialized Kernels

The specialized kernels for symmetric matrices are **symm** and **syrk**, both of which are specialized versions of matrix multiplication involving symmetric matrices.

symm performs

$$C := \alpha AB + \beta C$$

or

$$C := \alpha BA + \beta C$$

where α and β are scalars, A is a symmetric matrix, and B and C are m by n matrices. **symm** also takes **side** and **uplo** parameters, which specify whether the symmetric matrix A is on the left or right and whether to read the upper or lower triangular matrix from memory, respectively. In subsequent sections, we will show how to leverage these two parameters to make memory layout optimizations.

Similarly, **syrk** performs

$$C := \alpha AA^T + \beta C$$

or

$$C := \alpha A^T A + \beta C$$

where α and β are scalars and C is a symmetric matrix. **syrk** takes the same **uplo** parameter and a **trans** parameter, which, like **symm**, determines whether to compute the first or second option.

The analogous operation for triangular matrices is **trmm**, which computes

$$B := \alpha \cdot \text{op}(A)B$$

or

$$B := \alpha \cdot B \text{ op}(A)$$

It takes analogous **side**, **uplo**, and **trans** parameters (which determines whether to take the transpose of A or not). **trmm** additionally takes a **diag** parameter, which specifies whether the matrix has a unit diagonal (in which case the diagonal will not be read from memory and assumed to be all ones).

For tridiagonal matrices, **gtsv** solves

$$AX = B$$

where A is a tridiagonal matrix, by Gaussian elimination with partial pivoting. To call the operation, A must be decomposed into its three diagonals, which are then passed as vectors to the kernel.

3.3 Mathematical Simplifications

A **TransposeOp** is a no-op on symmetric matrices.

An **LUFactorizationOp** (which performs an LU factorization with row pivoting) is a no-op on upper triangular matrices (but not necessarily on lower triangular matrices, because the rows may be swapped from the factorization process).

3.4 Lowering of Custom Calls

To support these specialized kernels on the compiler, we add custom ops to the Enzyme compiler’s MLIR dialect and write MLIR lowering passes to make the appropriate custom call depending on the architecture. In general, on CPU we make calls to the BLAS/LAPACK libraries, and on CUDA we make calls to the cuBLAS/cuSPARSE libraries. For TPU architectures or higher dimensional tensors (since typically these libraries only support operations on matrices) we write a fallback lowering that converts the custom op back into standard StableHLO operations.

To make the custom call, the lowering pass for our custom call primarily needs to compute the necessary additional parameters for the call. This typically includes the dimensions of the matrices, operand aliasing, and additional layout parameters. For CPU lowering, we additionally construct wrapper functions, which enables more flexibility on the specific kernel implementation used upon compiling.

3.5 Layout Optimizations

The Enzyme compiler assumes a row-major layout, and the BLAS, cuBLAS, etc. libraries assume a column-major layout. When making a custom call to a specialized kernel, the compiler must therefore specify whether or not the underlying matrix data needs to be transposed in order to fit the layout assumptions. Therefore, if we were to perform the lowering naively, the compiler would have to perform a transpose from row-major to column-major before making the custom call, then perform a transpose from column-major back into row-major to read the results of the custom call.

However, these memory layout transformations can be avoided through a set of neat tricks involving the parameters of each of these specialized kernels. The following sections will walk through how the compiler avoids changing memory layouts for each of the specialized kernels.

3.5.1 `symm` and `trmm`

`symm` computes symmetric matrix multiplication $\alpha AB + \beta C$, and `trmm` computes triangular matrix multiplication αAB . Both operations take `side` and `uplo` as parameters (see the previous section on supported specialized kernels or Appendix A for a fuller specification).

To avoid layout changes, we can compute everything in column-major space by swapping the `side` and `uplo` parameters. In column-major space, the matrix is transposed, so the upper triangular of the matrix becomes the lower triangular, and vice versa. Therefore, by swapping the `uplo` of the matrix, the custom call will reference the correct triangular of the matrix (note that for symmetric matrices, this triangular is then used to implicitly construct the symmetric matrix). For instance, if `uplo` was previously `UPPER`, reading from the upper triangular in row-major space is equivalent to reading from the lower triangular in column-major space, so we simply change `uplo` to `LOWER`.

Additionally, we know from basic linear algebra that $(AB)^T = B^T A^T$ for any matrix A and B . Note that A^T and B^T are the column-major interpretations of A and B respectively, and $(AB)^T$ is the column-major interpretation of the result of `symm` C . Therefore, we simply

swap the `side` parameter to compute the result in column-major space, which results in the correct result C in row-major space.

Therefore, the compiler does not need to perform any layout transformations (note that α and β are scalar, so these do not need to be transformed).

3.5.2 `syrk`

`syrk` performs $\alpha AA^T + \beta C$ where C is symmetric, and takes `trans` and `uplo`. We can again flip `trans` and `uplo`. The rationale for flipping `uplo` is the same, so we will omit it.

In column-major space, A becomes A^T and vice versa; by flipping `trans`, we can convert A^T back into A and vice versa. Then, since the output C is symmetric (and BLAS assumes the same output `uplo` as the input `uplo`) the compiler does not need to switch layout of C either (except for in cases of resolving differing `uplo` or materializing matrices).

3.5.3 `gtsv`

Since `gtsv` does not have any parameters, no analogous layout optimizations can be made for `gtsv`, so it suffers the performance hit from layout transformations.

3.6 Fusion of Addition and Multiplication

Since `symm`, `syrk`, and `trmm` operations are constructed from StableHLO `DotGeneral` operations, initially only the core matrix multiplication is captured by the first compiler pass. Additional compiler passes then need to be made to fuse any addition or multiplication operations into the corresponding custom operation, which reduces the total number of separate operations.

These fusion passes work by checking if an operand of an addition or multiplication operation is a valid custom op. If so, it will remove the add/mul op and add cheaper operations to recompute the corresponding α and β constants as necessary. For instance, if the compiler determines that a `MultiplyOp` is multiplying the result of a `SymmOp` by some constant θ , it will insert new `MultiplyOps` that perform the cheaper scalar operations $\theta \cdot \alpha_{old}$ and $\theta \cdot \beta_{old}$ to compute new α and β constants, then update the `SymmOp` to use those new coefficients. The original elementwise `MultiplyOp` can then be removed from the IR.


```

1 func.func @test1(%arg0: tensor<64x64xf32>, %arg1: tensor<64xf32>, %arg2: tensor<64
  x64xf32>) -> tensor<64x64xf32> {
2   %alpha = stablehlo.constant dense<2.0> : tensor<f32>
3   %beta = stablehlo.constant dense<3.0> : tensor<f32>
4   %scalar = stablehlo.constant dense<3.0> : tensor<f32>
5   %scalar_bcast = stablehlo.broadcast_in_dim %scalar, dims = []: (tensor<f32>) ->
    tensor<64x64xf32>
6   %0 = enzymexla.blas.symm %arg0, %arg1, %arg2, %alpha, %beta {side = #enzymexla.
    side<left>, uplo = #enzymexla.uplo<U>} : (tensor<64x64xf32>, tensor<64xf32>,
    tensor<64x64xf32>, tensor<f32>, tensor<f32>) -> tensor<64x64xf32>
7   %1 = stablehlo.multiply %scalar_bcast, %0 : tensor<64x64xf32>
8   return %1 : tensor<64x64xf32>
9 }

```

Figure 3.1: MLIR program using `symm` before fusion pass

```

1 func.func @test1(%arg0: tensor<64x64xf32>, %arg1: tensor<64xf32>, %arg2: tensor<64
  x64xf32>) -> tensor<64x64xf32> {
2   %[[cst2:..+]] = stablehlo.constant dense<2.000000e+00> : tensor<f32>
3   %[[cst3:..+]] = stablehlo.constant dense<3.000000e+00> : tensor<f32>
4   %0 = stablehlo.multiply %[[cst3]], %[[cst3]] : tensor<f32>
5   %1 = stablehlo.multiply %[[cst2]], %[[cst3]] : tensor<f32>
6   %2 = enzymexla.blas.symm %arg0, %arg1, %arg2, %1, %0 {side = #enzymexla.side<
    left>, uplo = #enzymexla.uplo<U>} : (tensor<64x64xf32>, tensor<64xf32>,
    tensor<64x64xf32>, tensor<f32>, tensor<f32>) -> tensor<64x64xf32>
7   return %2 : tensor<64x64xf32>
8 }

```

Figure 3.2: MLIR program using `symm` after fusion pass

Chapter 4

Evaluation

This section evaluates the compiler on a set of benchmarks that feature significant use of structured tensors. We look for successful detection of structure in each Julia program, and for calls to specialized kernels where possible.

The performance of our compiler will additionally heavily depend on the performance of the specialized external kernels that we emit. The performance of these kernels may vary by architecture, which will impact how much of a performance improvement the compiler can offer. This project only evaluates performance on one machine with an AMD CPU and the NVIDIA GeForce RTX 5090. On different architectures, results may vary.

4.1 Benchmarks

We first evaluate our compiler on a set of simple benchmarks involving structured tensors. Then, we evaluate the Newton-Schulz iteration, which is an algorithm frequently used in scientific computing and machine learning applications.

4.1.1 Basic Benchmarks

Figure 4.1.1 contains the Julia program for benchmarking triangular matrices; it simply takes the upper triangular of a matrix and returns the product of the triangular matrix and itself. The MLIR program generated from this Julia program is shown in Figure 4.1.1. It is clear from the annotations that the compiler successfully detects the triangular matrix from the IR. Line 9 shows the custom call to the `trmm` kernel, which is emitted because the compiler was able to infer that `%4` was upper triangular.

Figure 4.1.1 similarly takes a matrix and multiplies it by its transpose, creating a symmetric matrix, and performs more operations involving symmetric matrices.

For both benchmarks, structured tensor optimizations result in at least a 20% speed-up for large enough matrices. Figure 4.9 shows the relative speed-ups of all benchmarks for 4096×4096 matrices, and 4.8 shows the speed-ups by input size.

For smaller matrices it becomes apparent from Figure 4.8 that the overhead of these optimizations outweighs the speed-ups that the specialized kernels provide and the overall runtime is slightly worse. However, at these small sizes, the difference is less than a millisecond.

```

1 function f(A)
2     X = UpperTriangular(A)
3     return X * A * 2.0
4 end

```

Figure 4.1: Benchmark for triangular matrices

```

1 func.func @main(%arg0: tensor<256x256xf64> {enzymexla.memory_effects = []}, %arg1
  : tensor<256x256xf64> {enzymexla.memory_effects = []}) -> tensor<256x256xf64>
  attributes {enzymexla.memory_effects = []} {
2     %cst = stablehlo.constant dense<2.000000e+00> : tensor<f64>
3     %cst_0 = stablehlo.constant dense<0.000000e+00> : tensor<256x256xf64>
4     %0 = stablehlo.transpose %arg0, dims = [1, 0] {enzymexla.symmetric_matrix = [#
      enzymexla<guaranteed NOTGUARANTEED>]} : (tensor<256x256xf64>) -> tensor<256
      x256xf64>
5     %1 = stablehlo.iota dim = 0 : tensor<256x256xi64>
6     %2 = stablehlo.iota dim = 1 : tensor<256x256xi64>
7     %3 = stablehlo.compare LE, %1, %2 : (tensor<256x256xi64>, tensor<256x256xi64>)
      -> tensor<256x256xi1>
8     %4 = stablehlo.select %3, %0, %cst_0 {enzymexla.symmetric_matrix = [#enzymexla<
      guaranteed NOTGUARANTEED>], enzymexla.upper_tri_matrix = [#enzymexla<
      guaranteed GUARANTEED>], enzymexla.upper_unit_tri_matrix = [#enzymexla<
      guaranteed NOTGUARANTEED>]} : tensor<256x256xi1>, tensor<256x256xf64>
9     %5 = enzymexla.blas.trmm %4, %arg0, %cst {side = #enzymexla.side<right>,
      transpose = #enzymexla.transpose<transpose>, uplo = #enzymexla.uplo<U>} : (
      tensor<256x256xf64>, tensor<256x256xf64>, tensor<f64>) -> tensor<256
      x256xf64>
10    return %5 : tensor<256x256xf64>
11 }

```

Figure 4.2: MLIR for triangular tensor benchmark after compiler passes.

Because the sizes are known at compile time, it is also possible to disable these optimizations for small matrices; however, since the difference in runtime is so small, this version of the compiler does not implement this.

4.1.2 Optimizing the Newton-Schulz Iteration

One apt benchmark for symmetric tensors is the Newton-Schulz Orthogonalization Iteration, which is an algorithm that transforms either the columns or rows of a matrix into a set of orthonormal vectors. It is used in the Muon optimizer to perform fast, approximate orthogonalization of update matrices to accelerate the training of LLMs, and in many other scientific computing and machine learning applications. Figure 4.4 shows the implementation of the Newton-Schulz iteration in Julia. Note that in Julia, `'` indicates taking the transpose and `.` indicates broadcasting an operation elementwise.

```

1 function f(A, B)
2     X = A * A'
3     Y = X + B * B'
4     return Y * A * 2.0
5 end

```

Figure 4.3: Benchmark for symmetric matrices

```

1 function newton_schulz_muon(G; steps=12, eps=1e-A)
2     T = eltype(G)
3     a, b, c = (T(3.4445), T(-4.7750), T(2.0315))
4
5     # Normalize
6     X = G
7     X = X ./ (norm(X) + T(eps))
8
9     if size(G, 1) > size(G, 2)
10         X = X'
11     end
12
13     @trace for _ in 1:steps
14         A = X * X'
15         B = b .* A .+ c .* (A * A)
16         X = a .* X .+ B * X
17     end
18
19     if size(G, 1) > size(G, 2)
20         X = X'
21     end
22
23     return X
24 end

```

Figure 4.4: Implementation of the Newton-Schulz Iteration in Julia

It turns out that a significant portion of the Newton-Schulz iteration is computation involving symmetric matrices. The matrix A is symmetric as it is the product of X and X^T . Then, B is also symmetric, since AA is symmetric (as it is equivalent to $A^T A$, which we know to be symmetric), and the sum of two symmetric matrices must also be symmetric.

Given our set of possible optimizations for symmetric matrices, several of these operations can be rewritten as **syrk** and **symm** operations. Specifically, the multiplication XX^T and the entire computation for B can both be rewritten as **syrk** operations. Then, $aX + BX$ can be rewritten as a **symm** operation.

Figure 4.5 shows the core loop of the Newton-Schulz MLIR program after the detection and optimization passes are run on the program (but before the custom calls are lowered, for simplicity). Our compiler was able to infer which matrices in the core loop were guaranteed to be symmetric, as well as rewrite all three general matrix multiplications into specialized calls.

Figures 4.6 and 4.7 show the kernel statistics for Newton-Schulz before and after our compiler passes. In 4.6, all of the matrix multiplications are performed with a call to the generic kernel **gemm**. Figure 4.7 no longer contains any generic matrix multiplications, and instead only makes calls to the specialized matrix multiplication kernels **symm** and **syrk**.

From these figures, it also becomes apparent that in this case, **symm** strangely runs slower than **gemm**. It turns out that if the original tensor is fully materialized, **gemm** on CUDA is typically faster than **symm**, because **symm** is not yet as highly optimized. Therefore, in practice we have adjusted our optimization pass to only emit a **symm** call if the tensor is not yet fully materialized; however, we expect that as these specialized kernels get more and more optimized, the performance benefits will improve over time.

```

1 %A = enzymexla.blas.syrk %X, %c0, %c1, %c0 {enzymexla.symmetric_matrix = [#
    enzymexla<guaranteed GUARANTEED>], output_uplo = #enzymexla.uplo<F>, transpose
    = #enzymexla.transpose<transpose>, uplo = #enzymexla.uplo<F>} : (tensor<100
    x100xf64>, tensor<100x100xf64>, tensor<f64>, tensor<f64>) -> tensor<100x100xf64
    >
2 %bA = stablehlo.multiply %b, %A {enzymexla.symmetric_matrix = [#enzymexla<
    guaranteed GUARANTEED>]} : tensor<100x100xf64>
3 %B = enzymexla.blas.syrk %A, %bA, %c, %c1 {output_uplo = #enzymexla.uplo<U>, uplo =
    #enzymexla.uplo<F>} : (tensor<100x100xf64>, tensor<100x100xf64>, tensor<f64>,
    tensor<f64>) -> tensor<100x100xf64>
4 %aX = stablehlo.multiply %a, %X {enzymexla.symmetric_matrix = [#enzymexla<
    guaranteed NOTGUARANTEED>]} : tensor<100x100xf64>
5 %ret = enzymexla.blas.symm %B, %X, %aX, %c1, %c1 {side = #enzymexla.side<right>,
    uplo = #enzymexla.uplo<U>} : (tensor<100x100xf64>, tensor<100x100xf64>, tensor
    <100x100xf64>, tensor<f64>, tensor<f64>) -> tensor<100x100xf64>
6 stablehlo.return ... %ret : ... tensor<100x100xf64>

```

Figure 4.5: Newton-Schulz MLIR after detection and optimization passes, but before custom call lowering. Variables and constants have been renamed to better match what they represent. The compiler was able to successfully annotate symmetric matrices with `enzymexla.symmetric_matrix = [#enzymexla<guaranteed GUARANTEED>]`, which then allows the compiler to make the necessary rewrites into `symm` and `syrk`.

Kernel Name	Occurrences	Total Duration	Avg Duration	Min Duration	Max Duration
void dgemv_largek<true, false, 5, 5, 4, 4, 4, 34>(...	12	0.86569909s	0.07214159s	0.07213628s	0.07214585s
void dgemv_largek<false, true, 5, 5, 4, 4, 4, 32>(...	12	0.86565058s	0.07213755s	0.07212476s	0.07214380s
void dgemv_largek<false, false, 5, 5, 4, 4, 4, 32>...	12	0.86484427s	0.07207036s	0.07206178s	0.07207823s
void scal_kernel<double, double, 1, false, 6, 4, 4, ...	24	0.00410315s	0.00017096s	0.00016935s	0.00017319s
loop_multiply_fusion	12	0.00189698s	0.00015808s	0.00015712s	0.00015895s
loop_multiply_fusion_1	12	0.00189065s	0.00015755s	0.00015690s	0.00015799s
void scal_kernel<double, double, 1, true, 6, 4, 4, ...	12	0.00089064s	0.00007422s	0.00006797s	0.00007568s
loop_divide_fusion	1	0.00015891s	0.00015891s	0.00015891s	0.00015891s
input_reduce_fusion	1	0.00008886s	0.00008886s	0.00008886s	0.00008886s
loop_add_fusion	12	0.00003280s	0.00000273s	0.00000218s	0.00000294s
input_reduce_fusion_1	1	0.00000253s	0.00000253s	0.00000253s	0.00000253s
loop_add_fusion_1	1	0.00000128s	0.00000128s	0.00000128s	0.00000128s

Figure 4.6: Newton-Schulz kernel statistics without structured tensors detection. All matrix multiplications use the `gemm` (general matrix multiplication) kernel.

Kernel Name	Occurrences	Total Duration	Avg Duration	Min Duration	Max Duration
void cutlass::Kernel2<cutlass_80_tensorop_d884symm...	12	1.16377786s	0.09698149s	0.09697522s	0.09699129s
void cutlass::Kernel2<cutlass_80_tensorop_d884syrc...	12	0.64507960s	0.05375663s	0.05375411s	0.05376191s
void cutlass::Kernel2<cutlass_80_tensorop_d884syrc...	12	0.64461042s	0.05371753s	0.05371391s	0.05372051s
input_transpose_fusion	12	0.00266672s	0.00022223s	0.00021971s	0.00022436s
loop_multiply_fusion_1	12	0.00198467s	0.00016539s	0.00016352s	0.00016781s
loop_multiply_fusion	12	0.00178345s	0.00014862s	0.00014813s	0.00014954s
loop_divide_fusion	1	0.00016688s	0.00016688s	0.00016688s	0.00016688s
input_reduce_fusion	1	0.00008864s	0.00008864s	0.00008864s	0.00008864s
loop_add_fusion	12	0.00003209s	0.00000267s	0.00000176s	0.00000326s
loop_compare_fusion_1	1	0.00001661s	0.00001661s	0.00001661s	0.00001661s
input_reduce_fusion_1	1	0.00000266s	0.00000266s	0.00000266s	0.00000266s
loop_add_fusion_1	1	0.00000138s	0.00000138s	0.00000138s	0.00000138s

Figure 4.7: Newton-Schulz kernel statistics with structured tensors detection and optimization enabled. This run includes the call to `symm`; however since on this architecture `symm` is more expensive than `gemm` it would not replace `gemm` in practice.

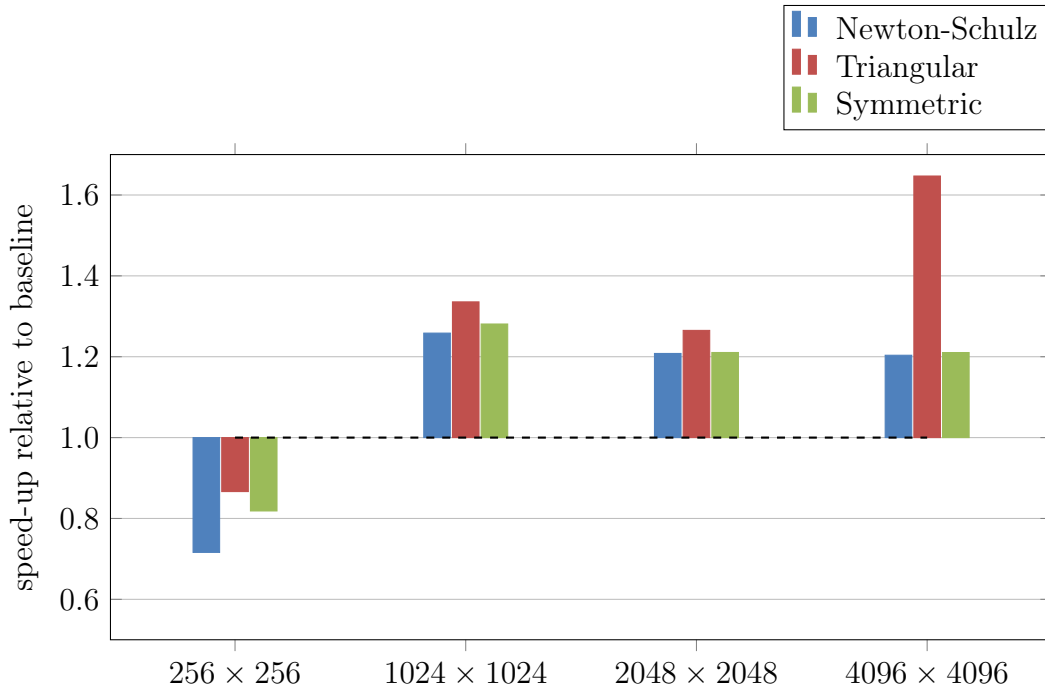


Figure 4.8: speed-up of structured tensor benchmarks over the default optimization options across different sizes of input matrices.

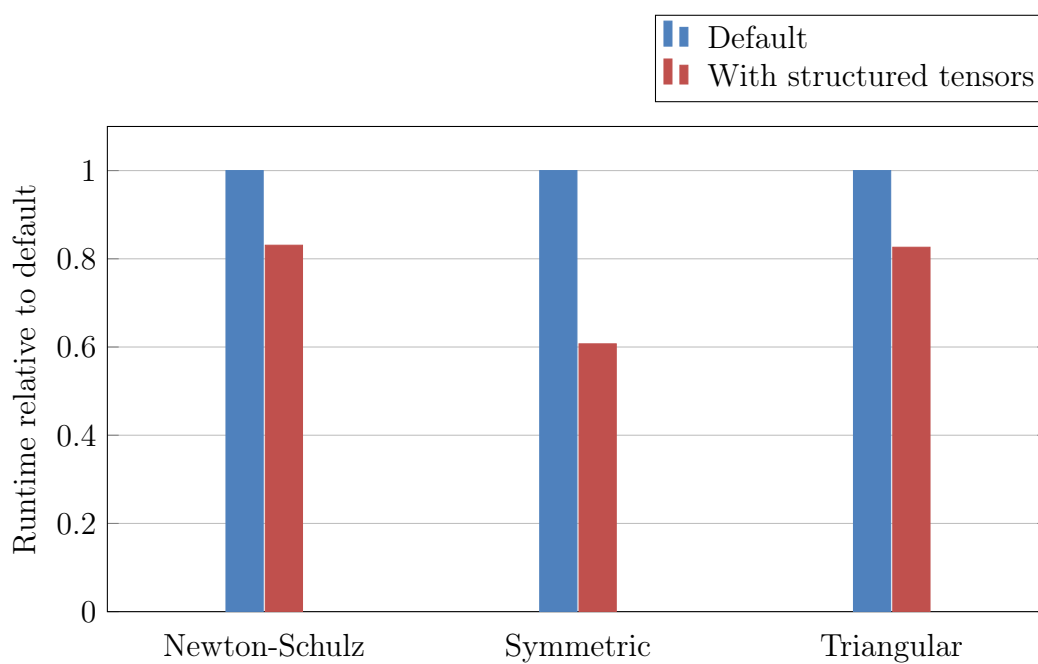


Figure 4.9: Relative speed-up of each benchmark relative to the default optimization options, with normalized default runtime.

Chapter 5

Conclusion

In this work, we presented a compiler framework for the automatic detection and optimization of structured tensors in high-level programs. Our evaluation demonstrates that this approach yields meaningful performance improvements across structured workloads, at no additional effort from the programmer.

More broadly, our compiler highlights the importance of preserving mathematical semantics in modern compiler design. As machine learning and scientific computing increasingly rely on high-level tensor abstractions, there is growing opportunity to bridge the gap between mathematical intent and low-level execution. Operating on top of these high-level intermediate representations allows us to more easily and robustly make inferences about data patterns than previous approaches based on more complex control or data flow analyses.

There are a few different possibilities for future work – one is to expand the range of supported structures and corresponding optimizations, including composed structure and value properties. Another is to integrate cost models and autotuning strategies to dynamically select among multiple specialized implementations, which becomes particularly important as the performance of these structure-aware kernels varies based on the underlying architecture.

Overall, our results suggest that structure-aware compilation can serve as a foundation for the next generation of high-performance tensor compilers.

Appendix A

Full MLIR program for Newton-Schulz

Listing A.1: Full lowering of Newton-Schulz iteration

```
1  module @reactant_newton_... attributes {mhlo.num_partitions = 1 : i64, mhlo.  
    num_replicas = 1 : i64} {  
2  func.func @main(%arg0: tensor<100x100xf64> {enzymexla.memory_effects = []}) ->  
    tensor<100x100xf64> attributes {enzymexla.memory_effects = []} {  
3    %cst = stablehlo.constant {enzymexla.finite = [#enzymexla<guaranteed GUARANTEED  
        >], enzymexla.no_nan = [#enzymexla<guaranteed GUARANTEED>]} dense<2.031500e  
        +00> : tensor<f64>  
4    %cst_0 = stablehlo.constant dense<3.444500e+00> : tensor<100x100xf64>  
5    %cst_1 = stablehlo.constant {enzymexla.symmetric_matrix = [#enzymexla<  
        guaranteed GUARANTEED>]} dense<-4.775000e+00> : tensor<100x100xf64>  
6    %cst_2 = stablehlo.constant dense<1.000000e+00> : tensor<f64>  
7    %cst_3 = stablehlo.constant dense<0.000000e+00> : tensor<100x100xf64>  
8    %c = stablehlo.constant dense<0> : tensor<i64>  
9    %c_4 = stablehlo.constant dense<12> : tensor<i64>  
10   %c_5 = stablehlo.constant dense<1> : tensor<i64>  
11   %cst_6 = stablehlo.constant dense<9.9999999999999995E-8> : tensor<f64>  
12   %cst_7 = stablehlo.constant {enzymexla.finite = [#enzymexla<guaranteed  
        GUARANTEED>], enzymexla.no_nan = [#enzymexla<guaranteed GUARANTEED>]} dense  
        <0.000000e+00> : tensor<f64>  
13   %0 = stablehlo.dot_general %arg0, %arg0, contracting_dims = [0, 1] x [0, 1] : (  
        tensor<100x100xf64>, tensor<100x100xf64>) -> tensor<f64>  
14   %1 = stablehlo.sqrt %0 : tensor<f64>  
15   %2 = stablehlo.add %1, %cst_6 : tensor<f64>  
16   %3 = stablehlo.broadcast_in_dim %2, dims = [] : (tensor<f64>) -> tensor<100  
        x100xf64>  
17   %4 = stablehlo.divide %arg0, %3 {enzymexla.symmetric_matrix = [#enzymexla<  
        guaranteed NOTGUARANTEED>]} : tensor<100x100xf64>  
18   %5:2 = stablehlo.while(%iterArg = %c, %iterArg_8 = %4) : tensor<i64>, tensor  
        <100x100xf64> attributes {enzyme.disable_mincut, enzymexla.symmetric_matrix  
        = [#enzymexla<guaranteed UNKNOWN>, #enzymexla<guaranteed UNKNOWN>, #  
        enzymexla<guaranteed UNKNOWN>, #enzymexla<guaranteed UNKNOWN>, #enzymexla<  
        guaranteed UNKNOWN>, #enzymexla<guaranteed UNKNOWN>, #enzymexla<guaranteed
```

```

UNKNOWN>, #enzymexla<guaranteed NOTGUARANTEED>]]}
19 cond {
20   %6 = stablehlo.compare LT, %iterArg, %c_4 : (tensor<i64>, tensor<i64>) ->
      tensor<i1>
21   stablehlo.return %6 : tensor<i1>
22 } do {
23   %6 = stablehlo.add %iterArg, %c_5 {enzymexla.bounds = [[1, 12]]} : tensor<i64>
      >
24   %7 = enzymexla.blas.syrk %iterArg_8, %cst_3, %cst_2, %cst_7 {enzymexla.
      symmetric_matrix = [#enzymexla<guaranteed GUARANTEED>], output_uplo = #
      enzymexla.uplo<F>, transpose = #enzymexla.transpose<transpose>, uplo = #
      enzymexla.uplo<F>} : (tensor<100x100xf64>, tensor<100x100xf64>, tensor<f64>
      >, tensor<f64>) -> tensor<100x100xf64>
25   %8 = stablehlo.multiply %cst_1, %7 {enzymexla.symmetric_matrix = [#enzymexla<
      guaranteed GUARANTEED>]]} : tensor<100x100xf64>
26   %9 = enzymexla.blas.syrk %7, %8, %cst, %cst_2 {output_uplo = #enzymexla.uplo<
      U>, uplo = #enzymexla.uplo<F>} : (tensor<100x100xf64>, tensor<100x100xf64>
      >, tensor<f64>, tensor<f64>) -> tensor<100x100xf64>
27   %10 = stablehlo.multiply %cst_0, %iterArg_8 {enzymexla.symmetric_matrix = [#
      enzymexla<guaranteed NOTGUARANTEED>]]} : tensor<100x100xf64>
28   %11 = enzymexla.blas.symm %9, %iterArg_8, %10, %cst_2, %cst_2 {side = #
      enzymexla.side<right>, uplo = #enzymexla.uplo<U>} : (tensor<100x100xf64>,
      tensor<100x100xf64>, tensor<100x100xf64>, tensor<f64>, tensor<f64>) ->
      tensor<100x100xf64>
29   stablehlo.return %6, %11 : tensor<i64>, tensor<100x100xf64>
30 }
31 return %5#1 : tensor<100x100xf64>
32 }
33 }

```

Appendix B

`symm` Lowering

The following is an example of a standard call to a BLAS kernel (in this case `symm`) and how it is lowered for CPU and CUDA architectures.

```

1 module {
2     func.func @main1(%arg0: tensor<64x64xf32>, %arg1: tensor<64x32xf32>, %arg2:
3         tensor<64x32xf32>) -> tensor<64x32xf32> {
4         %alpha = stablehlo.constant dense<2.0> : tensor<f32>
5         %beta = stablehlo.constant dense<3.0> : tensor<f32>
6         %0 = enzymexla.blas.symm %arg0, %arg1, %arg2, %alpha, %beta {side = #
7             enzymexla.side<left>, uplo = #enzymexla.uplo<U>} : (tensor<64x64xf32>,
8                 tensor<64x32xf32>, tensor<64x32xf32>, tensor<f32>, tensor<f32>) ->
9                 tensor<64x32xf32>
10        return %0 : tensor<64x32xf32>
11    }
12 }

```

Figure B.1: `symm` call to be lowered


```

1 func.func private @enzymexla_blas_ssymb_wrapper_[[SYMMID:[0-9]+]](%arg0: tensor<64
  x64xf32>, %arg1: tensor<64x32xf32>, %arg2: tensor<64x32xf32>, %arg3: tensor<f32
  >, %arg4: tensor<f32>) -> tensor<64x32xf32> {
2   %[[cSide:.+]] = stablehlo.constant dense<82> : tensor<ui8>
3   %[[cUplo:.+]] = stablehlo.constant dense<76> : tensor<ui8>
4   %[[c64:.+]] = stablehlo.constant dense<64> : tensor<i64>
5   %[[c32:.+]] = stablehlo.constant dense<32> : tensor<i64>
6   %0 = enzymexla.jit_call @enzymexla_blas_ssymb_wrapper ([cSide], %[[cUplo]],
    %[[c32]], %[[c64]], %arg3, %arg0, %[[c64]], %arg1, %[[c32]], %arg4, %arg2,
    %[[c32]]) {operand_layouts = [dense<> : tensor<0xindex>, dense<> : tensor<0
    xindex>, dense<> : tensor<0xindex>, dense<> : tensor<0xindex>, dense<[1, 0]> : tensor<2xindex>, dense<> : tensor<0xindex>,
    dense<[1, 0]> : tensor<2xindex>, dense<> : tensor<0xindex>, dense<> :
    tensor<0xindex>, dense<[1, 0]> : tensor<2xindex>, dense<> : tensor<0xindex
    >], output_operand_aliases = [#stablehlo.output_operand_alias<
    output_tuple_indices = [], operand_index = 10, operand_tuple_indices = []>],
    result_layouts = [dense<[1, 0]> : tensor<2xindex>], xla_side_effect_free} :
    (tensor<ui8>, tensor<ui8>, tensor<i64>, tensor<i64>, tensor<f32>, tensor
    <64x64xf32>, tensor<i64>, tensor<64x32xf32>, tensor<i64>, tensor<f32>,
    tensor<64x32xf32>, tensor<i64>) -> tensor<64x32xf32>
7   return %0 : tensor<64x32xf32>
8 }
9 llvm.func private @enzymexla_blas_ssymb_wrapper(%arg0: !llvm.ptr, %arg1: !llvm.
  ptr, %arg2: !llvm.ptr, %arg3: !llvm.ptr, %arg4: !llvm.ptr, %arg5: !llvm.ptr,
  %arg6: !llvm.ptr, %arg7: !llvm.ptr, %arg8: !llvm.ptr, %arg9: !llvm.ptr, %
  arg10: !llvm.ptr, %arg11: !llvm.ptr) {
10   %0 = llvm.mlir.constant(1 : i64) : i64
11   llvm.call @enzymexla_blas_ssymb_(%arg0, %arg1, %arg2, %arg3, %arg4, %arg5, %
    arg6, %arg7, %arg8, %arg9, %arg10, %arg11, %0, %0) : (!llvm.ptr, !llvm.ptr,
    !llvm.ptr, !llvm.ptr, !llvm.ptr, !llvm.ptr, !llvm.ptr, !llvm.ptr, !llvm.
    ptr, !llvm.ptr, !llvm.ptr, !llvm.ptr, i64, i64) -> ()
12   llvm.return
13 }
14 llvm.func @enzymexla_blas_ssymb_(!llvm.ptr, !llvm.ptr, !llvm.ptr, !llvm.ptr, !
  llvm.ptr, !llvm.ptr, !llvm.ptr, !llvm.ptr, !llvm.ptr, !llvm.ptr, !llvm.ptr, !
  llvm.ptr, i64, i64)
15 func.func @main1(%arg0: tensor<64x64xf32>, %arg1: tensor<64x32xf32>, %arg2:
  tensor<64x32xf32>) -> tensor<64x32xf32> {
16   %[[cst2:.+]] = stablehlo.constant dense<2.000000e+00> : tensor<f32>
17   %[[cst3:.+]] = stablehlo.constant dense<3.000000e+00> : tensor<f32>
18   %0 = call @enzymexla_blas_ssymb_wrapper_[[SYMMID:[0-9]+]](%arg0, %arg1, %arg2,
    %[[cst2]], %[[cst3]]) : (tensor<64x64xf32>, tensor<64x32xf32>, tensor<64
    x32xf32>, tensor<f32>, tensor<f32>) -> tensor<64x32xf32>
19   return %0 : tensor<64x32xf32>
20 }

```

Figure B.2: symm call lowered for CPU

```

1 func.func @main1(%arg0: tensor<64x64xf32>, %arg1: tensor<64x32xf32>, %arg2: tensor
  <64x32xf32>) -> tensor<64x32xf32> {
2   %0 = tensor.empty() : tensor<0xf32>
3   %1 = tensor.empty() : tensor<0xf32>
4   %2 = stablehlo.custom_call @enzymejax_cublas_symm_ffi(%arg0, %arg1, %arg2, %0,
    %1) {api_version = 4 : i32, backend_config = {alpha_imag = 0.000000e+00 :
    f64, alpha_real = 2.000000e+00 : f64, beta_imag = 0.000000e+00 : f64,
    beta_real = 3.000000e+00 : f64, side = false, uplo = true,
    use_alpha_attribute = true, use_beta_attribute = true}, operand_layouts = [
    dense<[1, 0]> : tensor<2xindex>, dense<[1, 0]> : tensor<2xindex>, dense<[1,
    0]> : tensor<2xindex>, dense<0> : tensor<1xindex>, dense<0> : tensor<1
    xindex>], output_operand_aliases = [#stablehlo.output_operand_alias<
    output_tuple_indices = [], operand_index = 2, operand_tuple_indices = []>],
    result_layouts = [dense<[1, 0]> : tensor<2xindex>]} : (tensor<64x64xf32>,
    tensor<64x32xf32>, tensor<64x32xf32>, tensor<0xf32>, tensor<0xf32>) ->
    tensor<64x32xf32>
5   return %2 : tensor<64x32xf32>
6 }

```

Figure B.3: `symm` call lowered for CUDA

References

- [1] W. Ahrens, T. F. Collin, R. Patel, K. Deeds, C. Hong, and S. Amarasinghe. “Finch: Sparse and Structured Tensor Programming with Control Flow.” *arXiv (Cornell University)*, Apr. 2024. DOI: <https://doi.org/10.48550/arxiv.2404.16730>.
- [2] C. Lattner, M. Amini, U. Bondhugula, A. Cohen, A. Davis, J. Pienaar, R. Riddle, T. Shpeisman, N. Vasilache, and O. Zinenko. “MLIR: Scaling Compiler Infrastructure for Domain Specific Computation.” In: *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. 2021, pp. 2–14. DOI: [10.1109/CGO51591.2021.9370308](https://doi.org/10.1109/CGO51591.2021.9370308).
- [3] A. Paszke et al. “PyTorch: An Imperative Style, High-Performance Deep Learning Library.” *CoRR*, **abs/1912.01703**, 2019. arXiv: [1912.01703](https://arxiv.org/abs/1912.01703). URL: <http://arxiv.org/abs/1912.01703>.
- [4] Apple Inc. *The Swift Programming Language*. Version 6.0. 2024. URL: <https://docs.swift.org/swift-book/documentation/the-swift-programming-language/>.
- [5] J. Bezanson, A. Edelman, S. Karpinski, and V. B. Shah. “Julia: A fresh approach to numerical computing.” *SIAM Review*, **59**(1), 2017, pp. 65–98. DOI: [10.1137/141000671](https://doi.org/10.1137/141000671). URL: <https://epubs.siam.org/doi/10.1137/141000671>.
- [6] T. S. Authors. *StableHLO*. OpenXLA Project. 2026. URL: <https://github.com/openxla/stablehlo>.
- [7] E. Anderson et al. *LAPACK Users’ Guide*. Third. Philadelphia, PA: Society for Industrial and Applied Mathematics, 1999. ISBN: 0-89871-447-8 (paperback).
- [8] W. Moses and V. Churavy. “Instead of Rewriting Foreign Code for Machine Learning, Automatically Synthesize Fast Gradients.” In: *Advances in Neural Information Processing Systems*. Vol. 33. Curran Associates, Inc., 2020, pp. 12472–12485.
- [9] W. S. Moses et al. “DJ4Earth: Differentiable, and Performanceportable Earth System Modeling via Program Transformations.”
- [10] F. Kjølstad, S. Y. Chou, D. Lugato, S. Kamil, and S. Amarasinghe. “Taco: A tool to generate tensor algebra kernels.” *DSpace@MIT (Massachusetts Institute of Technology)*, Oct. 2017. DOI: <https://doi.org/10.1109/ase.2017.8115709>.